Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «АПО ЭВМиС»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel – обработка символьных данных»

Выполнил:
Студент 4-го курса
Группы АС-66
Куган Н. Л.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: изучение программной модели микропроцессора, практическая работа с подсистемой регистровой и оперативной памяти.

Задание

Дан текст – непустая последовательность не длиннее ста символов. Признаком конца ввода является точка, в сам текст точка не входит. Проверить, удовлетворяет ли текст заданному условию. Если условие выполнено, преобразовать текст по указанному в варианте правилу; в противном случае выдать соответствующее сообщение. Преобразованный текст напечатать. Если введённая последовательность символов не является текстом, преобразовывать её не нужно, а следует напечатать соответствующее сообщение. Ввод текста, проверка условия, обработка текста и печать результата выполняются последовательно, отдельными частями программы (процедурами).

Вариант 9

Проверяемое условие: за каждой цифрой текста идёт строчная латинская буква.

Правило преобразования: заменить каждую прописную русскую букву симметричной ей в алфавите (А ↔ Я, Б ↔ Ю, …).

Реализация

Программа разработана в MS Visual Studio (консольное приложение C++, платформа x86) и состоит из двух файлов. В файле lab2_main.cpp выполнены ввод и вывод на C++, а проверка условия и преобразование текста реализованы на ассемблере (MASM) в отдельном файле lab2_asm.asm в виде процедур CheckConditionAsm и TransformTextAsm. Процедуры вызываются из C++ по соглашению cdecl и объявлены как extern "C". Функция ReadText читает текст до точки в буфер из 100 символов и определяет, что ввод не является текстом (пустой ввод, Enter до точки, более 100 символов). Консоль переключена в кодировку Windows-1251. Глобальные переменные не используются, копия текста не создаётся – текст преобразуется на месте.

Регистры: ESI – указатель на текущий символ текста, ECX – количество оставшихся символов, AL – текущий символ. Адрес текста и длина передаются

в процедуры через стек ([ebp+8] и [ebp+12]) и загружаются в ESI и ECX командами mov; регистр ESI, который по cdecl должен сохраняться, процедуры сохраняют командами push/pop. Результат проверки условия возвращается в регистре EAX (1 – условие выполнено, 0 – нет).

Проверка условия: текст просматривается слева направо; если встречена цифра (код 30h..39h), то она не должна быть последним символом, а следующий за ней символ должен лежать в диапазоне 61h..7Ah (a..z). Если цифр в тексте нет, условие выполняется.

Преобразование: прописные русские буквы в Windows-1251 занимают коды C0h..DFh (А..Я, 32 буквы, Ё имеет код A8h, в диапазон не входит и не изменяется). Симметричная буква вычисляется как код′ = DFh − (код − C0h), что даёт А(C0h) ↔ Я(DFh), Б(C1h) ↔ Ю(DEh) и т. д. В ассемблере это последовательность sub al, 0C0h; neg al; add al, 0DFh.

Текст программы

lab2_main.cpp

```
// lab2_main.cpp
// Лабораторная работа №2, вариант 9. Куган Н. Л., АС-66. Платформа: x86.
// Ввод текста, проверка условия, преобразование, вывод.
//
// Строки для проверки (вводить в консоль вместе с точкой):
//   ПРИВЕТ мир 5x ABC.      -> условие TRUE,  результат: РПЧЭЪН мир 5x ABC
//   Привет 7z, ЁЖ АЯБ.      -> условие TRUE,  результат: Рривет 7z, ЁЩ ЯАЮ
//   a1b2c.                  -> условие TRUE,  текст не меняется
//   ПРИВЕТ МИР.             -> условие TRUE (цифр нет), результат: РПЧЭЪН УЧП
//   abc 9.                  -> условие FALSE (цифра последняя)
//   12a.                    -> условие FALSE (за цифрой 1 идёт цифра 2)
//   .                       -> ошибка: пустой ввод
//   ТЕКСТ БЕЗ ТОЧКИ         -> ошибка: нет точки (Enter до точки)

#include <cstdio>
#include <windows.h>

#ifndef _M_IX86
#error Assembly module is built with ml.exe (x86 only): choose platform "x86" (Win32).
#endif

extern "C" int  CheckConditionAsm(const char* text, int len);   // реализация в lab2_asm.asm
extern "C" void TransformTextAsm(char* text, int len);          // реализация в lab2_asm.asm

const int MAX_LEN = 100;

int ReadText(char* buf, int& len)
{
    len = 0;
    for (;;)
    {
        int c = std::getchar();
        if (c == '.')
            break;
```

```
        if (c == '\n' || c == EOF)
            return 2;
        if (len >= MAX_LEN)
            return 2;
        buf[len++] = (char)c;
    }
    return (len == 0) ? 1 : 0;
}

int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    char text[MAX_LEN + 1] = { 0 };
    int len = 0;

    std::printf("============================================================\n");
    std::printf(" Lab 2, variant 9. Character data processing\n");
    std::printf(" Condition: every digit is followed by a lowercase\n");
    std::printf("            latin letter (a..z).\n");
    std::printf(" Rule     : every uppercase RUSSIAN letter is replaced\n");
    std::printf("            by its symmetric one (A<->YA, B<->YU, ...).\n");
    std::printf("============================================================\n\n");
    std::printf("Enter text (max 100 chars, end with '.'):\n> ");

    int status = ReadText(text, len);
    if (status == 1)
    {
        std::printf("\nError: empty input, this is not a text!\n");
        return 0;
    }
    if (status == 2)
    {
        std::printf("\nError: no '.' within 100 characters, this is not a text!\n");
        return 0;
    }
    std::printf("\nOriginal text   : %.*s\n", len, text);

    if (CheckConditionAsm(text, len) != 1)
    {
        std::printf("Condition check : FALSE.\nTransformation skipped.\n");
        return 0;
    }
    std::printf("Condition check : TRUE.\nApplying transformation...\n");

    TransformTextAsm(text, len);

    std::printf("Transformed text: %.*s\n", len, text);
    return 0;
}
```

lab2_asm.asm

```
; lab2_asm.asm
; Лабораторная работа №2 (ассемблер, обработка символьных данных). Вариант 9.
; Куган Н. Л., АС-66.
;
; Условие:
;     за каждой цифрой текста идёт строчная латинская буква (a..z).
;
; Правило преобразования (применяется только если условие истинно):
;     каждая прописная русская буква (кодировка CP1251, 0C0h..0DFh)
;     заменяется симметричной ей в алфавите: А<->Я, Б<->Ю, ...
;
; extern "C" int  CheckConditionAsm(const char* text, int len);
; extern "C" void TransformTextAsm(char* text, int len);
;
; Платформа проекта - x86 (Win32), сборка через ml.exe.


.386
.MODEL FLAT, C
.CODE


; ---------------------------------------------------------------
; int CheckConditionAsm(const char* text, int len)
; Возвращает 1 (истина) или 0 (ложь) в eax.
; ---------------------------------------------------------------
PUBLIC CheckConditionAsm
CheckConditionAsm PROC
    push    ebp
    mov     ebp, esp
    push    esi


    mov     esi, [ebp+8]         ; esi = text
    mov     ecx, [ebp+12]        ; ecx = len
cc_loop:
    test    ecx, ecx
    jz      cc_true              ; текст закончился - нарушений нет
    mov     al, [esi]
    inc     esi
    dec     ecx
    cmp     al, 30h              ; '0'
    jb      cc_loop
    cmp     al, 39h              ; '9'
    ja      cc_loop              ; не цифра - берём следующий символ


    test    ecx, ecx
    jz      cc_false             ; цифра - последний символ, после неё буквы нет
    mov     al, [esi]            ; al = символ сразу после цифры
    cmp     al, 61h              ; 'a'
    jb      cc_false
    cmp     al, 7Ah              ; 'z'
    ja      cc_false
    jmp     cc_loop              ; за цифрой строчная латинская - проверяем дальше


cc_true:
    mov     eax, 1
    jmp     cc_end
cc_false:
    xor     eax, eax
cc_end:
    pop     esi
    pop     ebp
    ret
CheckConditionAsm ENDP



; ---------------------------------------------------------------
; void TransformTextAsm(char* text, int len)
; ---------------------------------------------------------------
PUBLIC TransformTextAsm
TransformTextAsm PROC
    push    ebp
```

```
    mov     ebp, esp
    push    esi

    mov     esi, [ebp+8]        ; esi = text
    mov     ecx, [ebp+12]       ; ecx = len
tt_loop:
    test    ecx, ecx
    jz      tt_done
    mov     al, [esi]
    cmp     al, 0C0h            ; 'А' в CP1251
    jb      tt_next
    cmp     al, 0DFh            ; 'Я' в CP1251
    ja      tt_next
    sub     al, 0C0h            ; al = номер буквы в алфавите (0..31)
    neg     al
    add     al, 0DFh            ; al = 0DFh - номер = симметричная буква
    mov     [esi], al
tt_next:
    inc     esi
    dec     ecx
    jmp     tt_loop
tt_done:
    pop     esi
    pop     ebp
    ret
TransformTextAsm ENDP

END
```

Результаты работы

Программа собрана в Visual Studio (x86) и запущена; ниже приведены контрольные примеры.

№	Ввод	Результат	Проверяет
1	ПРИВЕТ мир 5x ABC.	Условие TRUE → «РПЧЭЪН мир 5x ABC»	правило 9
2	Привет 7z, ЁЖ АЯБ.	TRUE → «Рривет 7z, ЁЩ ЯАЮ»	границы А/Я, Ё не меняется
3	a1b2c.	TRUE, текст без изменений	цифры + латиница
4	abc 9.	FALSE (цифра последняя)	цифра в конце
5	12a.	FALSE (за «1» идёт «2»)	цифра после цифры
6	ПРИВЕТ МИР.	TRUE (цифр нет) → «РПЧЭЪН УЧП»	условие без цифр
7	«.»	ошибка: пустой ввод	не текст
8	101 символ + «.»	ошибка: нет точки в 100 символах	ограничение длины
9	ТЕКСТ БЕЗ ТОЧКИ + Enter	ошибка: нет точки	не текст

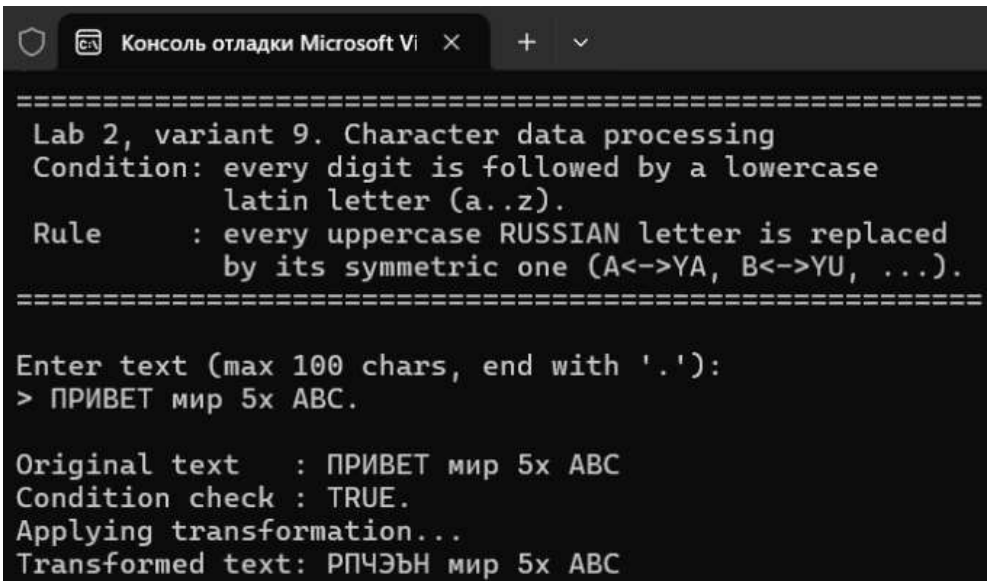


Рисунок 1 – Условие выполнено, текст преобразован

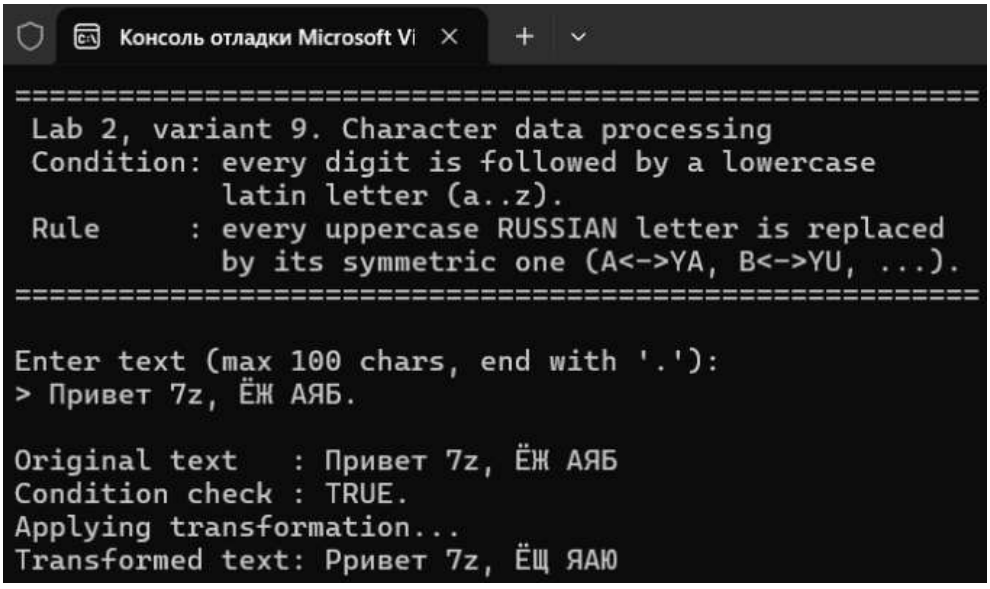


Рисунок 2 – Условие выполнено, проверка границ алфавита (А, Я, Ё)

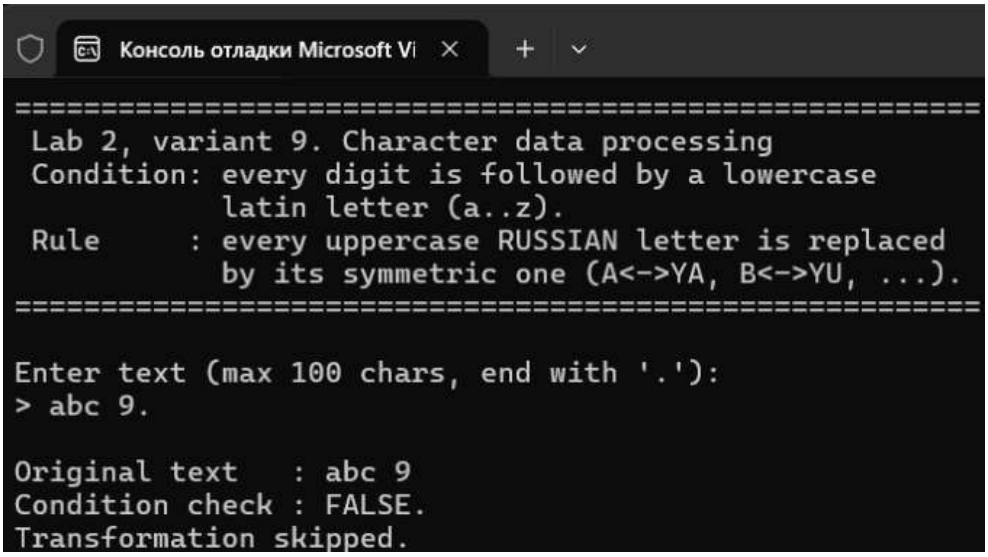


Рисунок 3 – Условие не выполнено (цифра в конце текста)

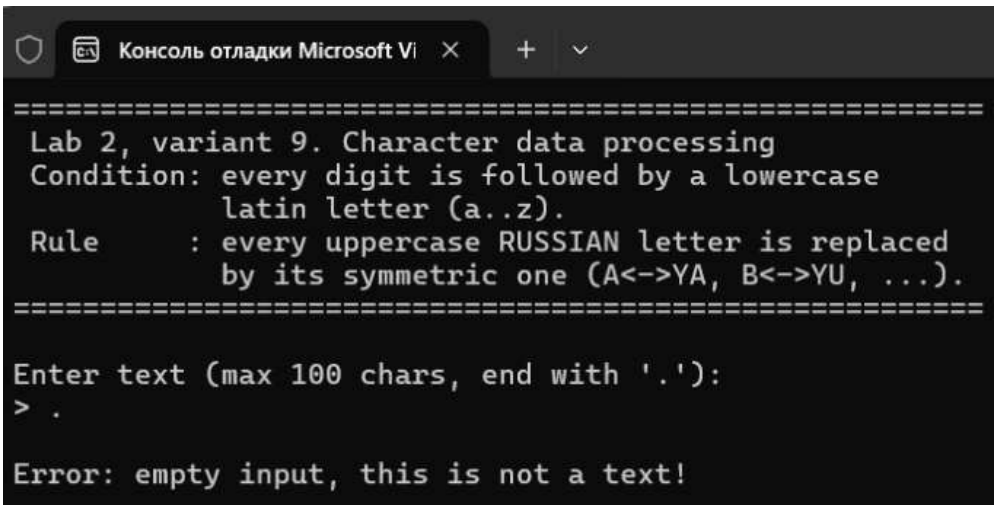


Рисунок 4 – Пустой ввод – не текст

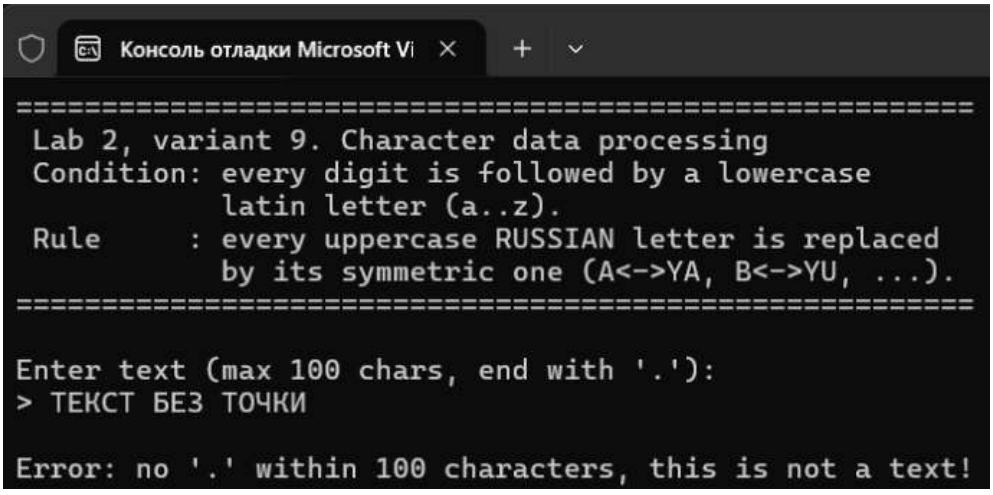


Рисунок 5 – Нет точки – не текст

Вывод: В ходе лабораторной работы изучена программная модель микропроцессора, получен практический опыт работы с регистрами и оперативной памятью при обработке символьных данных. Реализована программа с отдельным ассемблерным модулем для варианта 9: проверка условия «за каждой цифрой следует строчная латинская буква» и замена прописных русских букв симметричными в алфавите. Задача решена в виде отдельных процедур ввода (C++), проверки (CheckConditionAsm) и преобразования (TransformTextAsm), расположенных в файле lab2_asm.asm, без глобальных переменных и без использования дополнительной памяти, зависящей от длины текста.